SARTHI

Product and Engineering Guide

A repository-grounded reference to what the application is for, how its features work, what supports them, and where they can evolve.

> **Plan realistically. Focus on what matters today. Recover when life changes.**

Prepared from the current workspace source, documentation, tests, page templates, API routes, and data models. Private environment files and ignored personal seed data were excluded.

Repository snapshot: 11 November 2026

How to use this guide

This guide covers the product as both a user-facing application and a software system. Each feature explains its intended outcome, the user flow, the implementation evidence, and practical extension points. The repository contains a mix of working code, design specifications, seeded examples, and roadmap ideas; these are deliberately distinguished.

> **Evidence labels: Implemented means code is present in this repository. Documented intent means a product/design document defines the behavior. Planned/optional means the scope is described but a complete working integration is not evident. A visible page alone does not prove its backend workflow is complete.**

Contents

How to use this guide 2

Contents . . . 3

1. Product overview and design intent 7

The problem it is designed to solve . . . 7

Core principle . . . 7

Product workflow . . . 7

Behavioral guardrails . . . 7

2. Repository architecture and runtime 8

Local startup . . . 8

Connection behavior and operational caveat . . . 8

Terminal messages . . . 8

3. Navigation and feature map 9

4. Today dashboard: the execution home 10

Today / dashboard . . . 10

Why the page is deliberately short-horizon . . . 10

5. Tasks and backlog 11

Tasks & Backlog . . . 11

Task lifecycle and practical meaning . . . 11

6. Adaptive planner and recovery engine 12

Capacity-aware replanning . . . 12

Current ranking and allocation rules in PlannerEngine . . . 12

What to tune . . . 12

7. Timetable and calendar 13

Timetable templates . . . 13

Calendar . . . 13

8. Focus timer and work sessions 14

Focus . . . 14

9. Learning tracks: Web Development and DSA 15

Course progress engine . . . 15

Effort estimation . . . 15

DSA topic coverage . . . 15

10. Oratory programme 16

13-week Oratory . . . 16

11. Academics, attendance, assignments, and exams 17

Academics . . . 17

Attendance calculation . . . 17

Assignments and examinations . . . 17

12. Hackathons and projects 18

Hackathon command center . . . 18

13. Fitness, weight, and habits 19

Fitness and weight . . . 19

Habits and routine check-ins . . . 19

14. Journal, notes, and Reset Room 20

Journal and Notes . . . 20

Reset Room . . . 20

15. Reading, Exploration Lab, and Share Market Education 21

Business Reading . . . 21

Exploration Lab . . . 21

Share Market Education . . . 21

16. Savings, rewards, analytics, and settings 22

Savings and Rewards . . . 22

Analytics and Insights . . . 22

Settings, backup, and integrations . . . 22

Authentication . . . 22

Installable PWA shell . . . 23

Styleguide and theme . . . 23

17. Day-state engine and emotionally safe copy 24

Day state and behavior adaptation . . . 24

Copy Engine . . . 24

Change opportunities . . . 24

18. Time, dates, timezone, and display rules 25

Stored and displayed time . . . 25

Known coverage gap . . . 25

Timezone concerns . . . 25

19. API reference 26

20. Data model catalogue 27

Data integrity principles . . . 27

21. Seed data and curriculum content 28

Initial account seeding . . . 28

Source documents . . . 28

Safe changes . . . 28

22. Design system, accessibility, and current conflicts 29

Current code-level visual direction . . . 29

Documentation conflict . . . 29

Quality expectations 29

23. Security, privacy, and integrations 30

Security foundations present 30

Configuration and privacy 30

Integrations 30

24. Tests and verification map 31

25. Known gaps and prioritized improvement opportunities 32

26. Change guide by feature 33

27. Source inventory and provenance 34

Key files for future maintainers 34

1. Product overview and design intent

SARTHI is a personal execution and student operating system. It brings academic work, independent learning, projects, routines, and personal goals into one workspace with daily planning as the shared coordination layer.

The problem it is designed to solve

Students often split tasks, class schedules, attendance, course progress, project deadlines, health routines, notes, and goals across separate tools. The hard part is not merely storing these items. It is deciding what fits into limited time, what matters most, what should move when the day changes, and how to restart without turning one missed day into a backlog spiral.

Core principle

> **A missed day must never automatically invalidate the next day.**

Product workflow

Stage	Meaning
Capture	Record tasks, fixed commitments, learning work, and personal responsibilities.
Understand capacity	Use schedules, effort estimates, protected blocks, and daily limits to represent available time.
Plan	Create a manageable daily commitment and identify the next meaningful action.
Execute	Use focus sessions, task completion, partial progress, and check-ins to record real work.
Adapt	Replan transparently, defer work to bounded backlog, and preserve completed effort.
Review	Use history, analytics, and reflections to improve future estimates and routines.

Behavioral guardrails

- Avoid shame, threats, guilt, and red-screen failure states in supportive copy.
- Keep the Today view centered on the next action; put long-range plans and full backlog in dedicated views.
- Represent effort and progress honestly. Do not inflate minutes, completion, or streaks.
- Treat rest and low-energy days as supported pathways, with the user retaining control.
- Do not automatically rewrite historical records when a schedule template changes.

2. Repository architecture and runtime

The project is a small Node.js application. Express serves static HTML pages and a JSON API; Mongoose connects the API to MongoDB. The frontend is made of HTML, CSS, and browser ES modules rather than a bundled SPA build.

Layer	Repository location	Role
Browser pages	client/pages/*.html	Twenty-three page entry points. Most page logic is embedded as inline module scripts.
Shared client assets	client/assets/css and client/assets/js	Design tokens, reset/layout/components, API client, navigation, auth, toast, and utility helpers.
Express app	server/app.js, server/server.js	Middleware, static hosting, API mount, route fallback, database startup, shutdown hooks.
Routes/controllers	server/routes, server/controllers	Endpoint definitions, authentication gates, validation, business actions, responses.
Services	server/services	Planner, day-state, attendance, copy, seeding, and course import logic.
Mongo models	server/models	Per-user task, plan, schedule, learning, academic, personal, and profile records.
PWA shell	client/manifest.webmanifest, client/service-worker.js	Install metadata and a cache-first static shell with API requests excluded.
Tests	tests/*.test.js	Unit tests for planner, state, attendance, time formatting, seeding, and content rules.
Product references	README.md, docs/	Product direction, timetable/curriculum references, emotional safety rules, and design guidance.

Local startup

- Install Node dependencies with npm install, then run npm run dev for the watch-mode server or npm start for normal startup.
- The default backend port is 5000. The server serves the client directly, so a separate Live Server is optional; serving the client elsewhere requires the API base URL logic in client/assets/js/api.js.
- The health endpoint is GET /api/health. It reports whether the Mongoose connection is active and includes the current database state.
- Environment configuration is loaded from .env. Use .env.example as the safe template; never include a real .env in documentation, commits, or support messages.

Connection behavior and operational caveat

server/config/db.js attempts MongoDB connection during startup. It logs a warning and still starts the HTTP server if database selection/authentication fails. Data-dependent routes are guarded by requireDB and return HTTP 503 when MongoDB is disconnected, avoiding long Mongoose query-buffer timeouts. A successful server start therefore does not by itself mean the database is ready.

> **A healthy API process and a healthy database are separate states. Confirm database.connected is true at /api/health before trying registration or other database-backed actions.**

Terminal messages

The database module uses ANSI colors for branded success and warning messages, detects literal angle-bracket URI placeholders, and prints a sanitized URI for that mistake. Dotenv reports environment loading; server/server.js reports app initialization, listening URL, environment, and timezone. Keep raw credentials out of logs, distinguish database readiness from HTTP readiness, and never claim a database connection when health is false.

3. Navigation and feature map

Page	Purpose	Main supporting domain
Today / dashboard	Choose and act on today's next useful task.	DailyPlan, Task, planner, day state
Tasks	Create, filter, update, defer, partially complete, and complete work.	Task
Planner	See capacity, recovery options, and replan proposals.	PlannerEngine, DailyPlan, SlackDayLog
Timetable	Edit recurring day templates and inspect conflicts.	ScheduleTemplate
Calendar	Browse month dates and time-bearing commitments.	Tasks, exams, date data
Focus	Run and record timed work sessions.	FocusSession, Task
Web development / DSA	Track course sections and lecture progress.	Course, courseSeeder
Oratory	Track the 13-week speaking practice programme.	OratoryWeek
Academics	Manage subjects, attendance, assignments, and exams.	AcademicSubject, AttendanceRecord, Assignment, Examination
Hackathons	Track project/competition stages and deadlines.	Hackathon
Fitness / Habits	Log workouts and weight; check in to routines.	FitnessLog, WeightLog, Habit, HabitLog
Journal / Reset Room	Capture notes/reflections and recovery actions.	Note, DailyPlan, emotional routes
Reading / Exploration / Share Market	Build optional mastery tracks, reading, and simulated finance learning.	ReadingLog, Course, MarketConcept
Rewards	Track savings and unlockable rewards.	SavingsEntry, Reward
Analytics	Review execution progress and summary measures.	Insights endpoint, task/focus history
Settings	Adjust preferences, inspect integrations, and export data.	Setting, integration status, export
Installable app shell	Install metadata and cached static app shell.	Web manifest, service worker cache
Styleguide	Preview design tokens, components, and theme modes.	Shared CSS system
Sign in	Register, authenticate, and access the personal workspace.	User, JWT, cookie

This feature map is based on page titles, endpoint registrations, models, and documentation. UI pages can contain mock/seed content or partial flows; check the implementation notes in each feature section before assuming production readiness.

4. Today dashboard: the execution home

Today / dashboard

Purpose	How it works in this repository
Reduce the number of decisions at the start of a work session. The user should see the next useful action, today's bounded commitments, and a low-friction way to record progress.	Load today's plan and tasks; show next action and scheduled blocks; let the user quick-add a task, start focus, complete work, or log partial progress. The dashboard has create-task and partial-progress modals and an empty state for a first task.

Implementation evidence: client/pages/dashboard.html; GET /api/planner/today; POST /api/tasks; POST /api/tasks/:id/partial and /complete; plannerController; DailyPlan and Task models.

Useful changes: Make Next Action the visual anchor; expose today's capacity and completed minutes without making the screen a report; provide clear late-start, returning, disrupted, and low-energy states; ensure backlog stays out of the main daily view.

Status / caveat: Rendered UI and supporting APIs exist. The exact dashboard adaptation across every documented day state should be checked page-by-page; the design specification is richer than some current code paths.

Why the page is deliberately short-horizon

The product documents describe 'Horizon Fog': keep later commitments visually quiet until they matter. The aim is to lower cognitive load, protect attention, and prevent a large roadmap from making the first step feel impossible.

5. Tasks and backlog

Tasks & Backlog

Purpose	How it works in this repository
Provide one dependable source for actionable work, distinct from a calendar-only list.	Create a task with title, priority, effort, optional deadline/dependencies/protection; filter by all, today, backlog, P1, or completed; update, defer, partially complete, complete, or delete.

Implementation evidence: client/pages/tasks.html; Task model; /api/tasks routes and taskController. Priority badges are created by client/assets/js/utils.js.

Useful changes: Add recurrence rules and explicit task dependency editing; improve validation, keyboard workflows, multi-select actions, and audit history; make 'defer' preserve a reason and target date when chosen.

Status / caveat: Task API includes partial completion and deferral endpoints. Confirm the exact fields visible in each form against the Task schema before extending the UI.

Task lifecycle and practical meaning

State/action	What it means
Backlog / unscheduled	Work is captured but has no assigned day because capacity or timing is unresolved.
Scheduled	A target date or plan placement has been selected.
Partial progress	Some actual minutes are banked; only the remaining effort should be reconsidered.
Completed	The task is closed. Remaining effort is zero in planner calculations.
Deferred	The task is deliberately removed from the immediate plan and may remain in the backlog.

6. Adaptive planner and recovery engine

Capacity-aware replanning

Purpose	How it works in this repository
Prevent one disruption from creating an impossible catch-up day, while preserving essential work and user choice.	Generate a proposal from today's scheduled work, missed/unfinished tasks, backlog, daily capacity, monthly Slack Day allowance, and maximum catch-up minutes. The proposal lists accepted/deferred work, reasons, warnings, capacity, and whether confirmation is needed.

Implementation evidence: server/services/plannerEngine.js; plannerController; GET /api/planner/today; POST /api/planner/replan; POST /api/planner/replan/confirm; GET /api/planner/slack-days; DailyPlan, Task, SlackDayLog.

Useful changes: Show a preview with before/after capacity and a reason per decision. Allow edit, confirm, or reject. Add dependency-aware capacity simulation across upcoming days and make fairness/deadline trade-offs visible.

Status / caveat: Current planner engine uses a default 60-minute catch-up buffer in its pure function. Confirm controller/settings wiring and date/time-zone assumptions before treating every documented rule as active.

Current ranking and allocation rules in PlannerEngine

- Sort by earlier deadline first; tasks with a deadline sort before those without one.
- Then rank P1 before P2 before P3, protected tasks ahead of non-protected tasks, and smaller remaining effort first.
- Remaining effort is max(0, estimatedMinutes - actualMinutes), except completed work has zero remaining effort.
- Missed tasks with unmet dependencies are held. Partial minutes are preserved and the remainder is reconsidered.
- P1 or protected work may exceed the effective limit, but the engine flags a critical overload that needs user attention.
- P2 can use bounded catch-up; overflow and P3 work are deferred. Deferred work is placed only into future days with enough capacity, otherwise it remains backlog.
- A requested Slack Day defers today's commitments for recovery when allowance remains and requires explicit confirmation.

What to tune

Daily capacity, max work minutes, catch-up allowance, monthly Slack Days, P1/P2/P3 semantics, dependency rules, deadline urgency, and the future-day distribution horizon should all be user-visible policy rather than hidden magic numbers. Keep each proposal explainable and preserve deliberate user overrides.

7. Timetable and calendar

Timetable templates

Purpose	How it works in this repository
Represent recurring fixed commitments and available work windows so planning starts from real capacity.	Choose weekday, Saturday, Sunday, holiday, exam, or custom template; inspect its time blocks; edit/save a template; inspect overlapping blocks via conflict detection.

Implementation evidence: client/pages/timetable.html; ScheduleTemplate model; /api/timetable/templates, /:dayType, PUT /:id, /conflicts.

Useful changes: Add one-off overrides, drag/reorder, recurring exceptions, linked tasks, protected sleep validation, and an explicit resolution flow for conflicting blocks. Preserve history when templates change.

Status / caveat: Default timetable docs note an insufficient weekday sleep window and an unresolved Sunday overlap between Exploration and Oratory. These are product warnings to keep visible, not silently choose for the user.

Calendar

Purpose	How it works in this repository
Give the user a month-level way to orient deadlines and planned commitments without replacing the daily execution view.	Navigate months, jump to today, inspect dated items and start times; format event times for readable scanning.

Implementation evidence: client/pages/calendar.html; client date helper; calendar currently imports formatTime12 for event display.

Useful changes: Add robust item-type filters, keyboard-accessible day details, date-specific overrides, and richer task/exam/hackathon milestone support.

Status / caveat: The page exists; verify which calendar item types are persisted and wired to real APIs before treating it as a full calendar integration. Google Calendar is optional and currently only status/config scaffolding is evident.

8. Focus timer and work sessions

Focus

Purpose	How it works in this repository
Make starting a concrete work interval easier and record actual effort for planning and review.	Select a preset duration (25, 45, 60, or 90 minutes), start/pause/resume/complete/cancel; view active session and history; associate a session with a task when available.

Implementation evidence: client/pages/focus.html; FocusSession model; /api/focus/active, /start, /pause, /resume, /complete, /cancel, /history.

Useful changes: Add resilient timer recovery after refresh/device sleep, clear elapsed-vs-remaining semantics, sound/notification preference, session-to-task attribution, and interruption notes.

Status / caveat: A browser countdown should not be treated as a server-accurate clock unless the saved timestamps are authoritative. The page formats session timestamps with formatTime12.

9. Learning tracks: Web Development and DSA

Course progress engine

Purpose	How it works in this repository
Turn long video playlists into editable learning work and distinguish watching from active practice.	Import sections and lessons from Markdown source files; track watched and code-done flags, problem counts, difficulty, confidence, notes, effort, and completion totals; add sections or update lecture progress.

Implementation evidence: Course model; courseSeeder.js parses WEB_DEVELOPMENT_COURSE_SOURCE.md and DSA_COURSE_SOURCE.md; /api/courses, /:slug, /:slug/lecture/:lectureId, /:slug/section; pages webdev.html and dsa.html.

Useful changes: Allow per-course estimate multipliers and planned dates, revision queues, evidence links, lesson tags, skip/optional status, and a computed weekly workload preview before generating tasks.

Status / caveat: The reference describes 36 Web Development sections and lecture numbering through 331, plus 136 DSA lectures. Keep exact source titles/durations editable and treat imported completion as a reconcilable snapshot.

Effort estimation

The curriculum reference recommends a configurable 1.6x effort estimate for coding-heavy Web Development lessons: total effort starts from video duration multiplied by 1.6, with video duration and hands-on practice tracked separately. This is an estimate, not a guarantee; tune from actual sessions.

DSA topic coverage

The DSA course spans programming foundations, arrays and binary search, sorting and strings, recursion/backtracking, linked lists, stacks/queues, trees/BSTs, and graphs. The supplied reference asks each lesson to track lecture number/title/duration, study estimate, video completion, independent code completion, attempted/solved problems, notes, revision, confidence, difficulty, status, and planned date.

10. Oratory programme

13-week Oratory

Purpose	How it works in this repository
Build speaking skill through a repeated practice and feedback loop, not passive reading alone.	Follow one weekly topic/skill, prepare a speech, record a first attempt, assess it, drill one issue, record a final attempt, and reflect. Track progress week-by-week.

Implementation evidence: OratoryWeek model and courseSeeder; /api/courses/oratory/weeks and PUT /oratory/weeks/:weekNumber; client/pages/oratory.html.

Useful changes: Add structured rubric scoring, audio/video links, reminders, progress comparison, and flexible rescheduling when a week is missed.

Status / caveat: Source programme specifies a two-hour sequence and resuming at the next week rather than restarting the whole programme. Default timetable contains a Sunday slot conflict that must be resolved by the owner.

Week	Topic / skill focus
1	Who Am I Without Marks? - structure and hooks
2	Ancient India - gestures I
3	Education System in India - gestures II
4	Capitalism vs Socialism - voice modulation
5	Shivaji Maharaj and Sambhaji Maharaj - pauses
6	Religion in Modern Society - storytelling
7	History of Rajasthan - stage movement
8	Youth of India - eye contact
9	Business success and failure - persuasion
10	Feminism - explaining complex topics simply
11	Indian Independence - counterarguments
12	Arranged vs Love Marriage - charisma
13	Political Systems and Future of India - integration

11. Academics, attendance, assignments, and exams

Academics

Purpose	How it works in this repository
Keep course attendance and assessment work visible beside independent learning, with separate theory/practical accounting.	Create subjects; mark each attendance event with date, session type, and status; inspect attendance history; create/update assignments; create examinations; switch exam mode.

Implementation evidence: Academic routes/controller; AcademicSubject, AttendanceRecord, Assignment, Examination, and Setting models; client/pages/academics.html.

Useful changes: Add a what-if calculator (classes needed to reach target, safe absences), conflict notifications, assignment evidence, exam revision plans, and clearer audit/correction for attendance marks.

Status / caveat: Attendance engine is covered by tests. Thresholds and target percentages are defaults and should remain editable. Cancelled and exempted sessions need transparent denominator handling.

Attendance calculation

Subjects carry separate theory and practical totals and attended counts, plus minimum and target percentages. Attendance records capture the date, type, status, and notes. The attendance engine calculates current percentage and forward-looking attendance requirements; exact calculation policy should be reviewed alongside its tests before changing treatment of cancellations or exemptions.

Assignments and examinations

Assignments support due date, estimated effort, priority, status from assigned through graded, submission link, and marks. Examinations carry their own date/time and may be used to activate exam mode. Exam mode is stored in Settings with a date window and changes day-state selection.

12. Hackathons and projects

Hackathon command center

Purpose	How it works in this repository
Turn a competition/project into a sequence of visible stages and deadlines instead of an unbounded side project.	Create a hackathon/project, update its stage and details, and delete it. The product brief intends milestones, delegation, deadlines, and retrospective notes.

Implementation evidence: Hackathon model; /api/hackathons GET/POST/PUT/DELETE; hackathonController; client/pages/hackathons.html.

Useful changes: Add milestones and task links, team/member responsibilities, judging criteria, submission links, budget, calendar sync, and a structured post-event retrospective.

Status / caveat: The route/controller evidence supports CRUD; richer project workflow should be described as intent unless the current UI and schema implement it.

13. Fitness, weight, and habits

Fitness and weight

Purpose	How it works in this repository
Keep lightweight evidence of movement and body-weight trends without turning the app into a calorie or medical system.	Record workouts/fitness entries and dated weight measurements; view history and trend information on the fitness page.

Implementation evidence: FitnessLog and WeightLog models; /api/personal/fitness and /weight GET/POST; client/pages/fitness.html.

Useful changes: Add user-selected cadence, chart smoothing and units, editable baseline/goal, private export controls, and a neutral sick-day state. Keep medical guidance outside the app's scope.

Status / caveat: Product references explicitly avoid calorie counting and frame weight goals as user-entered rather than medical advice.

Habits and routine check-ins

Purpose	How it works in this repository
Make small repeated routines visible without creating a punitive streak system.	Review habit list and create a dated check-in against a habit; default seeded examples include night routine, learning, meditation, journaling, and no-sugar check-in.

Implementation evidence: Habit and HabitLog models; /api/personal/habits and /habits/checkin; client/pages/habits.html; default seeding in courseSeeder.

Useful changes: Support schedules/frequency, skip reasons, gentle trend summaries, custom habits, and a separate explicit rest/low-energy action so a miss is not mislabeled.

Status / caveat: Seeded habit labels are starter examples. Make them editable and avoid presenting them as medical advice.

14. Journal, notes, and Reset Room

Journal and Notes

Purpose	How it works in this repository
Provide private reflection space and searchable long-form notes outside the pressure of task completion.	Create and update notes; associate titles, content, dates, and tags where supported; browse the journal/notes page.

Implementation evidence: Note model; /api/personal/notes GET/POST/PUT; client/pages/journal.html.

Useful changes: Add search, autosave, archive/export, privacy lock expectations, and clearer separation between date-linked journal entries and evergreen notes.

Status / caveat: Private reflections should never be quoted in automatic nudges or surfaced outside their intended context.

Reset Room

Purpose	How it works in this repository
Help the user pause, choose a smaller next step, take a rest day, or record disruption without shame.	The UI is framed around pausing/regrouping; emotional endpoints support low-energy day, approved rest, close-day, profile, evidence, and feedback actions.

Implementation evidence: client/pages/reset-room.html; /api/emotional routes; DailyPlan and UserProfile; content rules and dayState engine.

Useful changes: Keep choices explicit and reversible; offer one small next action, an optional rest path, and a private reflection. Explain any streak or plan effect before confirming.

Status / caveat: Some behavior is documented more fully in docs/EMOTIONAL_LAYER.md than it is necessarily surfaced across every page.

15. Reading, Exploration Lab, and Share Market Education

Business Reading

Purpose	How it works in this repository
Convert business books into reading sessions, notes, and practical takeaways.	Browse core reading references, create reading session logs, and review progress.

Implementation evidence: ReadingLog model; /api/personal/reading GET/POST; client/pages/reading.html.

Useful changes: Add page/chapter targets, note capture, takeaway-to-task actions, search, and a configurable reading plan.

Status / caveat: Books such as SPIN Selling and Influence are starter references; keep catalog and goal editable.

Exploration Lab

Purpose	How it works in this repository
Let the user create a flexible mastery track for skills that do not fit the fixed Web Development or DSA courses.	Create a track, add topics, mark progress, and capture synthesis notes; customize track names and content.

Implementation evidence: client/pages/explore.html; Course model supports an exploration slug; course routes include add section and lecture progress.

Useful changes: Make track CRUD semantics explicit, add outcome/evidence fields, export/import, and connect selected topics to weekly capacity.

Status / caveat: Confirm persistent coverage for custom track operations against the course schema/controllers before expanding the UI.

Share Market Education

Purpose	How it works in this repository
Teach market concepts and provide safe practice records without becoming a brokerage or advice product.	Browse concept content and record paper/simulated practice against a concept.

Implementation evidence: MarketConcept model; /api/personal/market GET and /:conceptId/paper POST; client/pages/sharemarket.html.

Useful changes: Add a clear simulated-only label, glossary search, scenario notes, risk concepts, and performance reflection without personalized buy/sell prompts.

Status / caveat: The product reference explicitly excludes brokerage functionality and personalized investment recommendations.

16. Savings, rewards, analytics, and settings

Savings and Rewards

Purpose	How it works in this repository
Connect a long-term reward to manageable contributions and milestones rather than all-or-nothing motivation.	Review rewards and savings progress; log a contribution; inspect progress toward a target.

Implementation evidence: Reward and SavingsEntry models; /api/personal/rewards-savings and /savings/contribute; client/pages/rewards.html.

Useful changes: Support editable goals, contribution history corrections, milestone dates, and transparent unlock criteria; never revoke earned progress for one missed day.

Status / caveat: Seed data contains example reward ideas and a savings target; these should remain editable and are not financial advice.

Analytics and Insights

Purpose	How it works in this repository
Help the user compare planned vs actual work and notice patterns that support better estimates.	Load a transparent summary of tasks/focus/execution; show update time and progress information.

Implementation evidence: client/pages/analytics.html; GET /api/insights/summary; insightsRoutes and related models.

Useful changes: Show definitions and denominators for every metric; add weekly/monthly comparison, data-quality cues, and filters without ranking or shaming the user.

Status / caveat: Use only verifiable data. Avoid treating task counts alone as meaningful progress.

Settings, backup, and integrations

Purpose	How it works in this repository
Let the user control personal settings and retain a portable backup of their data.	Read/update settings; download a JSON export spanning account and product records; inspect Google Calendar and Web Push configuration status.

Implementation evidence: Setting model; /api/settings GET/PUT, /export, /integrations/status; settingsController; client/pages/settings.html.

Useful changes: Add import/restore with validation, integration consent/revoke controls, timezone preferences, notification schedules, and a clear local-data retention explanation.

Status / caveat: Integration status is not the same as a completed integration. Current settings controller reports credentials/readiness; do not imply Calendar sync or push delivery works without end-to-end implementation.

Authentication

Purpose	How it works in this repository
Scope personal records to the signed-in user and protect account access.	Register or log in; server hashes passwords, issues a JWT, and sets an HTTP-only cookie; authenticated routes use middleware; logout clears the cookie.

Implementation evidence: User model; authController; /api/auth/register, /login, /logout, /me; client/assets/js/auth.js and api.js.

Useful changes: Add account recovery, email verification, session revocation, stronger production secret validation, and consistent cookie/token strategy. Keep auth errors clear without leaking account existence unnecessarily.

Status / caveat: Registration seeds initial user data. Database connectivity must be confirmed for auth routes; the API health endpoint remains publicly reachable.

Installable PWA shell

Purpose	How it works in this repository
Make the workspace installable and keep the static application shell available when the network is unreliable.	The manifest defines the app name, standalone display mode, start URL, colors, and SVG icon. The service worker pre-caches selected shared files, claims the client, removes older caches, and serves cached GET assets; API requests and non-GET requests bypass that cache.

Implementation evidence: client/manifest.webmanifest; client/service-worker.js; page bootstrapping and shared navigation should be checked for service-worker registration.

Useful changes: Verify registration on every supported entry page, align manifest colors with the current light theme, version cache keys, add update UX, handle cache-add failures, and document offline limitations. Do not queue mutations without conflict-safe design.

Status / caveat: The service worker does not implement an offline write queue. Its fetch fallback can return the cached root for failed static GETs, so confirm route-specific offline behavior before promising full offline mode.

Styleguide and theme

Purpose	How it works in this repository
Keep pages visually consistent and provide a deliberate component reference.	The styleguide previews typography, colors, buttons, status signals, and theme controls. Shared CSS defines canvas/surfaces, blue/green/amber accents, spacing, radii, and shadows.

Implementation evidence: client/pages/styleguide.html; client/assets/css/variables.css, typography.css, reset.css, layout.css, components.css.

Useful changes: Treat the user's requested light look as default; use subtle bespoke gradients selectively, keep most surfaces solid, make accents semantic, and validate contrast, responsive states, and theme persistence.

Status / caveat: Conflict: docs/DESIGN_SYSTEM.md still describes a dark Night Workshop palette and bans most gradients, while current variables.css and styleguide use a light theme. Update docs to match the active product decision.

17. Day-state engine and emotionally safe copy

Day state and behavior adaptation

Purpose	How it works in this repository
Choose a useful tone and plan for the user's present condition rather than assuming every day is ordinary.	Compute a deterministic state using exam settings, rest/low-energy/disruption flags, active focus session, recent history, schedule timing, and planned progress. Emotional routes allow state changes and feedback.

Implementation evidence: server/services/dayStateEngine.js; /api/emotional/day-state, low-energy, rest-day, close-day, profile, evidence, feedback; UserProfile and DailyPlan.

Useful changes: Use the configured timezone consistently; test daylight and midnight boundaries; make state precedence visible; ensure the UI changes materially for each supported state.

Status / caveat: docs/EMOTIONAL_LAYER.md specifies fresh, in_flow, on_track, late_start, partial, disrupted, low_energy, rest_approved, exam_mode, returning, and milestone. Verify each state is actually reachable and displayed.

Copy Engine

The copy engine selects contextual messages from seeded content, interpolates limited user/context tokens, and records thumbs-up/down feedback to adjust message weight. Content guardrails reject shame and guilt language, protect private personal topics, prohibit flooding the Today view with backlog, and require honest arithmetic. These guardrails are tested in contentRules.test.js.

Change opportunities

- Make the copy engine fallback behavior explicit when no eligible message exists.
- Keep feedback opt-in, avoid sensitive inference, and provide a way to reset personalization.
- Add UI-level checks that templates preserve guardrails after interpolation.

18. Time, dates, timezone, and display rules

Stored and displayed time

ScheduleTemplate stores startTime/endTime as 24-hour strings such as HH:mm. The browser utility and server helper both provide formatTime12 and formatTimeRange, which convert those values to readable uppercase AM/PM output and handle noon and midnight. The client has adopted the helper on Dashboard, Timetable, Calendar, Academics, Focus, Analytics, and Settings where the search found usage.

Known coverage gap

The helpers are duplicated in client/assets/js/utils.js and server/utils/timeFormat.js. Several screens use them, but time-input controls and any direct time string construction must still be audited. Native browser time inputs commonly expose 24-hour values internally; preserve their HH:mm value for storage and format adjacent visible labels in 12-hour time. Never turn an AM/PM display string into the API's stored value.

Timezone concerns

Timezone defaults are configured as Asia/Kolkata, while some date calculations use JavaScript UTC or local Date methods. Before changing date logic, inventory every API/browser boundary and specify whether each value is a date-only string, local wall-clock time, or absolute timestamp. The shared formatter maps 00:00 to 12:00 AM, 12:00 to 12:00 PM, and 14:30 to 2:30 PM. Add boundary tests for midnight, noon, DST in non-India zones, and device timezone mismatch where those cases apply.

19. API reference

All data routes are mounted under /api. Most domain routes use authentication middleware. requireDB protects data-backed APIs and returns 503 when the database is disconnected; /health remains available for diagnosis.

Method	Endpoint path(s)	Access	Role
GET	/health	Public	API and MongoDB status
POST / POST / GET	/auth/register | /auth/login | /auth/me	Auth varies	Register, login, current user
POST	/auth/logout	Auth route	Clear session cookie
GET / POST / PUT / DELETE	/tasks	Authenticated	Task list and CRUD
POST	/tasks/:id/partial | /complete | /defer	Authenticated	Progress, completion, defer
GET / POST	/planner/today | /planner/replan | /planner/replan/confirm	Authenticated	Daily plan and recovery
GET	/planner/slack-days	Authenticated	Slack Day usage/status
GET / PUT	/timetable/templates | /templates/:dayType | /templates/:id	Authenticated	Schedule templates
GET	/timetable/conflicts	Authenticated	Overlap detection
GET / POST / PUT	/courses, /courses/:slug, /:slug/lecture/:id, /:slug/section	Authenticated	Course and lecture tracking
GET / PUT	/courses/oratory/weeks, /oratory/weeks/:weekNumber	Authenticated	Oratory programme
GET / POST / PUT	/academic(s)/subjects, /attendance, /attendance/history, /assignments, /exams, /exam-mode	Authenticated	Academic records
GET / POST / PUT / DELETE	/hackathons	Authenticated	Project/competition records
GET / POST	/focus/active, /start, /pause, /resume, /complete, /cancel, /history	Authenticated	Focus sessions
GET / POST	/personal/fitness, /weight, /habits, /habits/checkin, /notes, /reading, /market, /rewards-savings	Authenticated	Personal tracking reads/writes
PUT / POST	/personal/notes, /market/:conceptId/paper, /savings/contribute	Authenticated	Personal records and simulations
GET / POST / PUT	/emotional/day-state, /low-energy, /rest-day, /close-day, /profile, /evidence, /feedback	Authenticated	Day-state and reflection actions
GET / PUT	/settings, /settings/integrations/status	Authenticated	Preferences and integration status
GET	/settings/export	Authenticated	Full JSON export
GET	/insights/summary	Authenticated	Execution summary

Note: route aliases /api/academic and /api/academics both mount the academic router. The compact table groups related paths; consult server/routes for exact parameters and HTTP verbs before writing integrations.

20. Data model catalogue

The database is organized around per-user records. Domain collections generally include userId references, allowing user-scoped queries. Below is a functional catalogue, not a substitute for the authoritative Mongoose schemas.

Model	Main data	Product role
User	Identity, email, password hash, role, avatar, timestamps	Authentication root
Setting	Timezone, work limits, slack allowance, exam mode, personal preferences	Per-user operational policy
UserProfile	Day-state/copy profile, evidence, streak and goal preferences	Behavioral preferences; sensitive context needs strict privacy
Task	Title, priority, effort, actual minutes, deadline, status, dependencies, protection	Task lifecycle and planner input
DailyPlan	Date, day type, capacity, allocated/completed minutes, planned task snapshots and state flags	Daily commitment and recovery state
SlackDayLog	Slack day use by user/date and reason	Monthly recovery buffer history
ScheduleTemplate	Day type, named blocks, times, durations, category, protection	Recurring capacity model
FocusSession	Task/title, start/end/pause state, duration, session status	Actual execution evidence
Course	Slug, instructor, sections/lectures, effort, completion, confidence, notes	Learning track progress
OratoryWeek	Week number, topic, skill, draft/recording/scores/reflection fields	13-week speaking programme
AcademicSubject	Code/name/type, faculty/room, attendance thresholds and counts	Subject-specific attendance
AttendanceRecord	Subject, date, theory/practical type, present/absent/cancelled/exempted, notes	Attendance history
Assignment	Subject, title, due date, effort, priority/status, link, marks	Coursework tracking
Examination	Subject, exam date/time, type and details	Assessment schedule and exam mode
Hackathon	Competition/project attributes, dates, stage and notes	Project tracking
FitnessLog	Workout/activity record and date	Fitness history
WeightLog	Dated measurement and notes	Weight trend evidence
Habit	Habit definition, category, frequency, time of day	Routine catalogue
HabitLog	Habit/date check-in and status	Habit evidence
Note	Title/content/date/tags	Journal, notes, reflection
ReadingLog	Reading session, material, progress, notes	Book/reading activity
MarketConcept	Educational concept and simulated practice references	Financial literacy only
Reward	Reward definition, category, unlock criteria	Motivation catalogue
SavingsEntry	Goal, target, contributions	Savings progress
Message	Seeded contextual copy, eligibility and feedback weights	Context-sensitive prompts

Data integrity principles

- Scope every database query and update to the authenticated user wherever a user-owned record is involved.
- Keep date-only values in a stable format and time values separate from timestamps.
- Preserve completed records and actual effort when replanning or editing schedules.
- Do not expose password hashes, secrets, private notes, or personal profile details in diagnostics.

21. Seed data and curriculum content

Initial account seeding

Registration calls seedUserData for a new account. The seeder creates missing settings, default timetable templates, Web Development and DSA courses parsed from Markdown, 13 Oratory weeks, starter habits, a savings entry, and reward examples. Seeding is guarded by existence checks for most records so repeat calls do not intentionally duplicate each collection.

Source documents

- docs/source/WEB_DEVELOPMENT_COURSE_SOURCE.md contains the Web Development lesson source arranged in 36 sections, with lesson numbering through 331.
- docs/source/DSA_COURSE_SOURCE.md provides the 136-lecture DSA source and topic progression.
- docs/source/ORATORY_PROGRAM_SOURCE.md contains the 13-week oratory programme and associated tables.
- docs/COURSE_CURRICULA.md and docs/DEFAULT_TIMETABLE.md explain how the source materials should be imported, edited, and scheduled.

Safe changes

Treat source lesson titles and durations as editable imported material. Keep personal progress reconcilable; do not assume old completion snapshots are current. Keep seed files free of credentials or private profile exports. The .gitignore excludes .env, .env.* except .env.example, and server/seed/profile.seed.json.

22. Design system, accessibility, and current conflicts

Current code-level visual direction

client/assets/css/variables.css defines a light default with white surfaces, slate text, blueprint blue, botanical green, amber milestone accents, and an optional night theme. It uses a small spacing scale, compact radii, restrained shadows, and semantic color tokens. The styleguide page describes a polished light theme.

Documentation conflict

docs/DESIGN_SYSTEM.md is stale relative to the CSS: it describes a dark Night Workshop palette and strictly prohibits most gradients, while the current CSS is light and the user's requested direction is a personalized light theme with selective, intentional gradients. The PWA manifest also still declares dark navy background/theme colors despite the light CSS. Update the design specification, manifest colors, and page-specific hardcoded colors as one coherent design-system change.

Quality expectations

- Use semantic tokens instead of page-specific hard-coded colors; audit inline styles as well as shared CSS.
- Keep clear focus-visible rings, readable contrast, reduced-motion behavior, and touch targets of at least 44 by 44 CSS pixels where applicable.
- Use gradients as rare, meaningful surfaces; keep everyday cards and tables calm and legible.
- Support narrow screens, empty/loading/error states, and theme persistence consistently.

23. Security, privacy, and integrations

Security foundations present

- Helmet security middleware, JSON/form body limits, compression, cookie parsing, and CORS middleware are configured in server/app.js.
- Auth routes are rate-limited; user passwords are hashed with bcryptjs; JWTs are signed by server-side configuration; cookies are HTTP-only.
- Authenticated route groups call authenticate middleware, while the health check remains public for process/database diagnosis.
- Settings export omits the password hash from the exported user document and includes product data collections.

Configuration and privacy

.env and .env.* are ignored by Git except .env.example. The private profile seed is also ignored. Never paste credentials into docs, screenshots, logs, prompts, or commits. Production use should require explicit non-default secrets, secure cookies, a precise allowed-origin policy, and clear data deletion/export behavior.

Variable	Purpose (values are intentionally omitted)
PORT	HTTP server port; defaults to 5000.
NODE_ENV	Runtime mode; development is the default.
MONGODB_URI	MongoDB local or Atlas connection URI. Keep it private and single-line.
JWT_SECRET / SESSION_SECRET	Signing/session secrets; use strong, unique production values.
APP_TIMEZONE / TZ	Application timezone setting; initial app default is Asia/Kolkata.
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET / GOOGLE_REDIRECT_URI	Optional Google integration configuration.
VAPID_PUBLIC_KEY / VAPID_PRIVATE_KEY / VAPID_SUBJECT	Optional Web Push configuration.

Integrations

Settings reports whether Google Calendar credentials and Web Push VAPID keys are configured. This is readiness/status scaffolding; a status response alone is not proof of completed OAuth, event synchronization, subscription storage, delivery scheduling, or unsubscribe behavior. Treat each integration as incomplete until its full consent-to-delivery flow is implemented and verified.

24. Tests and verification map

Test file	Primary coverage indicated by filename/source
tests/plannerEngine.test.js	Priority, remaining effort, capacity, deferral, Slack Day/recovery proposal behavior.
tests/dayStateEngine.test.js	Day-state conditions and precedence.
tests/attendanceEngine.test.js	Attendance percentage and forward-looking calculations.
tests/timeFormat.test.js	12-hour conversion, noon/midnight, ranges, empty values.
tests/courseSeeder.test.js	Curriculum parsing and seed structures.
tests/contentRules.test.js	Copy content safety rules.

Package scripts include npm test. This guide documents test coverage from filenames and visible test source; no suite was run while producing this reference. Before changing a module, run its targeted tests and perform a browser smoke check for its full workflow.

25. Known gaps and prioritized improvement opportunities

Priority	Opportunity	Why it matters / acceptance signal
P0	Require database readiness before dependent operations; provide clear startup/health output.	Health says connected=true before auth or planning is tested; disconnected routes return immediate 503, not a query-buffer timeout.
P0	Consolidate duplicate client/server time format helpers and audit every time display/input.	Every visible time is AM/PM; stored schedule values remain HH:mm; tests cover all boundaries.
P0	Resolve design doc/CSS/styleguide contradictions and remove page-level hard-coded theme colors.	A single light theme source of truth controls all pages and user-preferred optional modes.
P1	Finish planner transparency and confirmation UI.	Every deferral/overload has a reason and the user can edit/confirm/reject the proposal.
P1	Improve timezone/date-only handling.	Local day boundaries agree across browser, server, and configured timezone.
P1	Clarify actual versus planned integration support.	Calendar/Push controls only promise flows that are implemented end-to-end.
P1	Add route-level validation and user-scope regression coverage.	Invalid inputs return structured 4xx; one account cannot read or mutate another account's records.
P2	Add accessible empty/loading/error and keyboard states across all 23 pages.	Core workflows remain usable on mobile, keyboard, and reduced-motion settings.
P2	Add import/restore, auditability, and safer per-domain exports.	Users can migrate and correct data without silent loss or surprise exposure.
P2	Add personalizable metric definitions and review cadence.	Analytics explain their inputs and avoid unsupported value judgments.

26. Change guide by feature

If you change...	Review these contracts first
Time format	Client/server formatters, native inputs, date-only vs timestamp semantics, schedule storage, formatting tests.
Planner ranking	PlannerEngine tests, actualMinutes preservation, deadlines, dependencies, P1 overload, P2 catch-up, backlog placement, confirm flow.
Timetable	ScheduleTemplate enum values, seeded templates, conflict detection, day-type mappings, history preservation, timezone.
Course import	Markdown parsing assumptions, exact source titles/durations, section/lecture IDs, progress snapshots, seeder idempotence.
Attendance	AttendanceRecord status set, theory/practical split, cancelled/exempted semantics, controller aggregation, tests.
Day-state copy	State precedence, contentRules tests, private-topic policy, backlog visibility, wording after token interpolation.
Authentication	JWT claims/expiry, cookie flags, localStorage token behavior, rate limiter, route auth, data isolation.
Theme	CSS variables, components, inline page styles, styleguide, reduced motion, contrast, responsive viewports.
Data export	Every new model/domain collection, password hash removal, JSON size, sensitive profile/notes, filename/version.

27. Source inventory and provenance

This guide was prepared from the repository workspace on 11 November 2026. It used the README, docs including design/timetable/curriculum/emotional-layer references, all page titles and a feature-page inventory, API routes/controllers, services, models, tests, shared CSS/JS, package metadata, and .gitignore. It did not read or reproduce .env secrets, node_modules internals, Git object contents, or the ignored personal profile seed.

Key files for future maintainers

Concern	Start here
Product intent	README.md; docs/EMOTIONAL_LAYER.md
Current design tokens	client/assets/css/variables.css; client/pages/styleguide.html
Stale design constraints to reconcile	docs/DESIGN_SYSTEM.md
Time conversion	client/assets/js/utils.js; server/utils/timeFormat.js; tests/timeFormat.test.js
Daily planning	server/services/plannerEngine.js; server/controllers/plannerController.js
Day state and copy	server/services/dayStateEngine.js; server/services/copyEngine.js; server/routes/emotionalRoutes.js
Database and environment	server/config/db.js; server/config/env.js; .env.example
API surface	server/routes/api.js and server/routes/*Routes.js
Seeded learning and schedule content	server/services/courseSeeder.js; docs/source/*

When a source document and executable code disagree, first identify which one is authoritative for the requested change. Update the code, tests, UI, and documentation together so future contributors do not inherit contradictory product rules.